

STUDENT ACADEMIC PERFORMANCE AND IMPROVEMENT ANALYSIS USING PYTHON

1. Project Overview

Developed an interactive education analytics dashboard using Power BI to analyse student academic performance, study behaviour, attendance patterns, motivation levels, parental involvement, and improvement trends. The project helps to identify factors influencing student success, monitor academic progress, and support educational decision-making through interactive visualizations and business insights.

Project Objective

- To analyse student academic performance effectively
- To evaluate attendance and study habits
- To identify factors affecting student grades
- To analyse student improvement patterns
- To examine the impact of parental involvement and professor quality
- To provide interactive and meaningful insights using Power BI

2. Data Sources

- **Source Description:** [Dataset Link](#)
- **Timeline:** Data collected during May 2024
- **Domain:** Education

3. Problem Statement

Analysing student academic records and educational factors by cleaning, transforming, and visualizing the dataset using Python and Power BI to identify performance trends, attendance patterns, study behaviour, improvement opportunities, and key factors influencing academic success.

4. Attribute (Column / Feature) Details

Attribute Name	Data Type	Description
Age	Integer	Age of the student
Gender	String (Text)	Gender of the student
Parental Education	String (Text)	Education level of parents
Family Income	Float	Family income level
Previous Grades	String (Text)	Previous academic grades
Attendance	Integer	Attendance percentage
Class Participation	String (Text)	Participation level in class
Study Hours	Integer	Weekly study hours
Major	String (Text)	Student's field of study
School Type	String (Text)	Public or Private institution
Financial Status	String (Text)	Financial condition
Parental Involvement	String (Text)	Level of parental involvement
Educational Resources	String (Text)	Availability of resources
Motivation	String (Text)	Student motivation level
Self Esteem	String (Text)	Confidence level
Stress Levels	String (Text)	Stress level
School Environment	String (Text)	School Environment perception
Professor Quality	String (Text)	Quality of professors
Class Size	Integer	Number of students in class
Sleep Patterns	Integer	Daily sleep hours
Screen Time	Integer	Daily screen time
Lack of Interest	String (Text)	Interest in academics
Time wasted on Social Media	Integer	Social media usage
Grades	String (Text)	Final academic grade

5. Tools & Technologies

1. **Python:** Data cleaning, handling missing values, data transformation, data type conversion, and exploratory data analysis.
2. **Power BI:** Data modelling, DAX calculations, data visualization, and interactive dashboard creation.
3. **Pandas & NumPy:** Data manipulation, preprocessing and numerical analysis.
4. **Matplotlib & Seaborn:** Data visualization and exploratory data analysis.

6. Data Pre-Processing

Initial EDA (Python):

 `df.size`

... 241536

 `df.columns`

... Index(['Age', 'Gender', 'Parental_Education', 'Family_Income',
'Previous_Grades', 'Attendance', 'Class_Participation', 'Study_Hours',
'Major', 'School_Type', 'Financial_Status', 'Parental_Involvement',
'Educational_Resources', 'Motivation', 'Self_Esteem', 'Stress_Levels',
'School_Environment', 'Professor_Quality', 'Class_Size',
'Sleep_Patterns', 'Screen_Time', 'Lack_of_Interest',
'Time_Wasted_on_Social_Media', 'Grades'],
dtype='object')

 `df.describe()`

...

	Age	Attendance	Study_Hours	Class_Size	Sleep_Patterns	Screen_Time	Time_Wasted_on_Social_Media
count	9099.000000	9060.000000	9073.000000	9114.000000	9048.000000	9120.000000	9137.000000
mean	19.996703	84.908499	12.467761	60.238754	6.538904	6.031469	3.489329
std	1.406885	8.985221	4.613845	23.337209	1.720632	2.597880	1.704057
min	18.000000	70.000000	5.000000	20.000000	4.000000	2.000000	1.000000
25%	19.000000	77.000000	9.000000	40.000000	5.000000	4.000000	2.000000
50%	20.000000	85.000000	12.000000	61.000000	7.000000	6.000000	3.000000
75%	21.000000	93.000000	16.000000	80.000000	8.000000	8.000000	5.000000
max	22.000000	100.000000	20.000000	100.000000	9.000000	10.000000	6.000000

 `df.dtypes`

...

	0
Age	float64
Gender	object
Parental_Education	object
Family_Income	object

```
df.shape
```

```
(10064, 24)
```

```
df.info()
```

```
... <class 'pandas.core.frame.DataFrame'>
RangeIndex: 10064 entries, 0 to 10063
Data columns (total 24 columns):
#   Column                                Non-Null Count  Dtype
---  -
0   Age                                   9099 non-null   float64
1   Gender                               9085 non-null   object
2   Parental_Education                   9102 non-null   object
3   Family_Income                        9097 non-null   object
4   Previous_Grades                      9120 non-null   object
5   Attendance                           9060 non-null   float64
6   Class_Participation                  9098 non-null   object
```

```
df.isna().sum()
```

	0
Age	965
Gender	979
Parental_Education	962
Family_Income	967
Previous_Grades	944
Attendance	1004
Class_Participation	966
Study_Hours	991
Major	928
School_Type	949

```
df.nunique()
```

	0
Age	5
Gender	2
Parental_Education	4
Family_Income	6
Previous_Grades	3
Attendance	31
Class_Participation	3
Study_Hours	16
Major	6
School_Type	2

```
df.duplicated().sum()
```

```
np.int64(0)
```

Python Data Cleaning & Transformation:

- Checked dataset for missing values
- Replaced missing numerical values using Mean Imputation
- Replaced missing categorical values using Mode Imputation
- Standardized categorical values and formats
- Verified data types and consistency
- No duplicate records were found in the dataset
- Prepared the dataset for Power BI analysis

```
# Fill missing values in the Age column using the mean value  
df['Age'] = df['Age'].fillna(df['Age'].mean())
```

```
# Change the datatype of age column:  
df['Age'] = df['Age'].astype(int)
```

```
# Check the number of students in each Gender  
df['Gender'].value_counts().reset_index()
```

	Gender	count
0	F	4557
1	M	4528

```
# Fill missing values in the Gender column using mode  
df['Gender'] = df['Gender'].fillna(df['Gender'].mode()[0])
```

```
# Display the updated distribution after filling missing values  
df['Gender'].value_counts().reset_index()
```

	Gender	count
0	F	5536
1	M	4528

```
# Check the number of students in each Parental education

df['Parental_Education'].value_counts().reset_index()
```

	Parental_Education	count
0	Some College	2323
1	Graduate	2277
2	College	2261
3	High School	2241



```
# Fill missing values in the Parental_Education column using mode
```

```
df['Parental_Education'] = df['Parental_Education'].fillna(df['Parental_Education'].mode()[0])
```

```
# Display the updated distribution after filling missing values
```

```
df['Parental_Education'].value_counts().reset_index()
```

	Parental_Education	count
0	Some College	3285
1	Graduate	2277
2	College	2261
3	High School	2241



```
# Convert Family_Income column to numeric format and replace invalid values with NaN
```

```
df['Family_Income'] = pd.to_numeric(df['Family_Income'], errors = 'coerce')
```

```
# Fill missing values in the Family_Income column using mean
```

```
df['Family_Income'] = df['Family_Income'].fillna(df['Family_Income'].mean())
```

```
# Check the number of students in each Previous Grade category
```

```
df['Previous_Grades'].value_counts().reset_index()
```

	Previous_Grades	count
0	C	3097
1	A	3036
2	B	2987



```
# Fill missing values in the Previous grade column using mode
```

```
df['Previous_Grades'] = df['Previous_Grades'].fillna(df['Previous_Grades'].mode()[0])
```

```
# Display the updated distribution after filling missing values
```

```
df['Previous_Grades'].value_counts().reset_index()
```

	Previous_Grades	count
0	C	4041
1	A	3036
2	B	2987

```
# Fill missing values in the Family_Income column using mean
```

```
df['Attendance'] = df['Attendance'].fillna(df['Attendance'].mean())
```

```
# Change the datatype of Attendance column
```

```
df['Attendance'] = df['Attendance'].astype(int)
```

```
# Check the number of students in each Class participation
```

```
df['Class_Participation'].value_counts().reset_index()
```

	Class_Participation	count
0	Medium	3047
1	Low	3026
2	High	3025

```
# Fill missing values in Class participation using the mode value
```

```
df['Class_Participation'] = df['Class_Participation'].fillna(df['Class_Participation'].mode()[0])
```

```
# Display the updated distribution after filling missing values
```

```
df['Class_Participation'].value_counts().reset_index()
```

	Class_Participation	count
0	Medium	4013
1	Low	3026
2	High	3025

```
# Fill missing values in Study hours column using the mean value
```

```
df['Study_Hours'] = df['Study_Hours'].fillna(df['Study_Hours'].mean())
```

```
# Change the datatype of Study hours column
```

```
df['Study_Hours'] = df['Study_Hours'].astype(int)
```

```
# Check the number of students in each Major
```

```
df['Major'].value_counts().reset_index()
```

	Major	count
0	Business	1576
1	Law	1569
2	Arts	1516
3	Science	1507
4	Medicine	1494
5	Engineering	1474

```
# Fill missing values in Major using the mode value
```

```
df['Major'] = df['Major'].fillna(df['Major'].mode()[0])
```

```
# Display the updated distribution after filling missing values
```

```
df['Major'].value_counts().reset_index()
```

	Major	count
0	Business	2504
1	Law	1569
2	Arts	1516
3	Science	1507
4	Medicine	1494
5	Engineering	1474


```
# Check the number of students in each School type
```

```
df['School_Type'].value_counts().reset_index()
```

	School_Type	count
0	Public	4580
1	Private	4535



```
# Fill missing values in School type using the mode value
```

```
df['School_Type'] = df['School_Type'].fillna(df['School_Type'].mode()[0])
```

```
# Display the updated distribution after filling missing values
```

```
df['School_Type'].value_counts().reset_index()
```

	School_Type	count
0	Public	5529
1	Private	4535



```
# Check the number of students in each Financial status
```

```
df['Financial_Status'].value_counts().reset_index()
```

	Financial_Status	count
0	Low	3068
1	Medium	3038
2	High	3023



```
# Fill missing values in Financial status using the mode value
```

```
df['Financial_Status'] = df['Financial_Status'].fillna(df['Financial_Status'].mode()[0])
```

```
# Display the updated distribution after filling missing values
```

```
df['Financial_Status'].value_counts().reset_index()
```

	Financial_Status	count
0	Low	4003
1	Medium	3038
2	High	3023



```
# Check the number of students in each Parental involvement
df['Parental_Involvement'].value_counts().reset_index()
```

	Parental_Involvement	count
0	High	3117
1	Medium	3057
2	Low	2944



```
# Fill missing values in mode value
```

```
df['Parental_Involvement'] = df['Parental_Involvement'].fillna(df['Parental_Involvement'].mode()[0])
```

```
# Display the updated distribution after filling missing values
df['Parental_Involvement'].value_counts().reset_index()
```

	Parental_Involvement	count
0	High	4063
1	Medium	3057
2	Low	2944



```
# Check the number of students in each Educational resources
df['Educational_Resources'].value_counts().reset_index()
```

	Educational_Resources	count
0	Yes	4634
1	No	4495



```
# Fill missing values using mode value
```

```
df['Educational_Resources'] = df['Educational_Resources'].fillna(df['Educational_Resources'].mode()[0])
```

```
# Display the updated distribution after filling missing values
df['Educational_Resources'].value_counts().reset_index()
```

	Educational_Resources	count
0	Yes	5569
1	No	4495



```
# Check the number of students in each Motivation level
```

```
df['Motivation'].value_counts().reset_index()
```

	Motivation	count
0	High	3062
1	Medium	3031
2	Low	3001



```
# Fill missing values using mode value
```

```
df['Motivation'] = df['Motivation'].fillna(df['Motivation'].mode()[0])
```

```
# Display the updated distribution after filling missing values
```

```
df['Motivation'].value_counts().reset_index()
```

	Motivation	count
0	High	4032
1	Medium	3031
2	Low	3001



```
# Check the number of students in each Self esteem
```

```
df['Self_Esteem'].value_counts().reset_index()
```

	Self_Esteem	count
0	Low	3106
1	High	3041
2	Medium	3005



```
# Fill missing values using mode value
```

```
df['Self_Esteem'] = df['Self_Esteem'].fillna(df['Self_Esteem'].mode()[0])
```

```
# Display the updated distribution after filling missing values
```

```
df['Self_Esteem'].value_counts().reset_index()
```

	Self_Esteem	count
0	Low	4018
1	High	3041
2	Medium	3005



```
# Check the number of students in each Stress levels  
df['Stress_Levels'].value_counts().reset_index()
```

	Stress_Levels	count
0	Low	3087
1	Medium	3059
2	High	2999

```
# Fill missing values using mode value
```

```
df['Stress_Levels'] = df['Stress_Levels'].fillna(df['Stress_Levels'].mode()[0])
```

```
# Display the updated distribution after filling missing values
```

```
df['Stress_Levels'].value_counts().reset_index()
```

	Stress_Levels	count
0	Low	4006
1	Medium	3059
2	High	2999

```
# Check the number of students in each School environment
```

```
df['School_Environment'].value_counts().reset_index()
```

	School_Environment	count
0	Neutral	3083
1	Negative	3043
2	Positive	2980

```
# Fill missing values using mode value
```

```
df['School_Environment'] = df['School_Environment'].fillna(df['School_Environment'].mode()[0])
```

```
# Display the updated distribution after filling missing values
```

```
df['School_Environment'].value_counts().reset_index()
```

	School_Environment	count
0	Neutral	4041
1	Negative	3043
2	Positive	2980

```
# Check the number of students in each Professor quality
df['Professor_Quality'].value_counts().reset_index()
```

	Professor_Quality	count
0	Low	3081
1	Medium	3034
2	High	2999



```
# Fill missing values using mode value
```

```
df['Professor_Quality'] = df['Professor_Quality'].fillna(df['Professor_Quality'].mode()[0])
```

```
# Display the updated distribution after filling missing values
```

```
df['Professor_Quality'].value_counts().reset_index()
```

	Professor_Quality	count
0	Low	4031
1	Medium	3034
2	High	2999



```
# Fill missing values using mean value
```

```
df['Class_Size'] = df['Class_Size'].fillna(df['Class_Size'].mean())
```

```
# Change the datatype of Class size column
```

```
df['Class_Size'] = df['Class_Size'].astype(int)
```

```
# Fill missing values using mean value
```

```
df['Sleep_Patterns'] = df['Sleep_Patterns'].fillna(df['Sleep_Patterns'].mean())
```

```
# Change the datatype of Sleep patterns column
```

```
df['Sleep_Patterns'] = df['Sleep_Patterns'].astype(int)
```

```
# Fill missing values using mean value
```

```
df['Screen_Time'] = df['Screen_Time'].fillna(df['Screen_Time'].mean())
```

```
# Change the datatype of Screen time column
```

```
df['Screen_Time'] = df['Screen_Time'].astype(int)
```

```
# Check the number of students in each Lack of interest
```

```
df['Lack_of_Interest'].value_counts().reset_index()
```

	Lack_of_Interest	count
0	High	3082
1	Medium	3050
2	Low	3004

```
# Fill missing values in Lack of interest using the mode value
```

```
df['Lack_of_Interest'] = df['Lack_of_Interest'].fillna(df['Lack_of_Interest'].mode()[0])
```

```
# Display the updated distribution after filling missing values
```

```
df['Lack_of_Interest'].value_counts().reset_index()
```

	Lack_of_Interest	count
0	High	4010
1	Medium	3050
2	Low	3004

```
# Fill missing values using the mean value
```

```
df['Time_Wasted_on_Social_Media'] = df['Time_Wasted_on_Social_Media'].fillna(df['Time_Wasted_on_Social_Media'].mean())
```

```
# Change the datatype of Time_Wasted_on_Social_Media column
```

```
df['Time_Wasted_on_Social_Media'] = df['Time_Wasted_on_Social_Media'].astype(int)
```

```
# Check the number of students in each grades
```

```
df['Grades'].value_counts().reset_index()
```

	Grades	count
0	B	3038
1	C	3010
2	A	2976

```
# Fill missing values in Grades using the mode value
df['Grades'] = df['Grades'].fillna(df['Grades'].mode()[0])
```

```
# Display the updated distribution after filling missing values
df['Grades'].value_counts().reset_index()
```

	Grades	count
0	B	4078
1	C	3010
2	A	2976

7. Data Modelling & DAX (Power BI)

Data Modelling:

- Established relationships between dataset fields to support analysis and reporting.
- Created calculated measures to monitor student performance and improvement trends.

DAX Measures Created:

- Total Students
- Average Study Hours
- Average Attendance
- Previous Grade C Students
- Students Improved (C to B)
- Improvement Rate (%)
- Students Not Improved

Total Students = `COUNTROWS(student_performance_cleaned)`

10064

Total Students

Average Study Hours = `AVERAGE(student_performance_cleaned[Study_Hours])`

12.42

Average Study Hours

Average Attendance = `AVERAGE(student_performance_cleaned[Attendance])`

84.82

Average Attendance

Previous Grade C =

```
CALCULATE(  
    COUNTROWS(student_performance_cleaned),  
    student_performance_cleaned[Previous_Grades] = "C"  
)
```

4041

Previous Grade C

Students Improved C to B =

```
CALCULATE(  
    COUNTROWS(student_performance_cleaned),  
    student_performance_cleaned[Previous_Grades]="C",  
    student_performance_cleaned[Grades]="B"  
)
```

1629

Students Improved C to B

```
Improvement Rate (%) = DIVIDE(  
    CALCULATE(COUNTROWS(student_performance_cleaned),  
        student_performance_cleaned[Previous_Grades]="C",  
        student_performance_cleaned[Grades]="B"),  
    CALCULATE(COUNTROWS(student_performance_cleaned),  
        student_performance_cleaned[Previous_Grades]="C"))
```

40.31%

Improvement Rate (%)

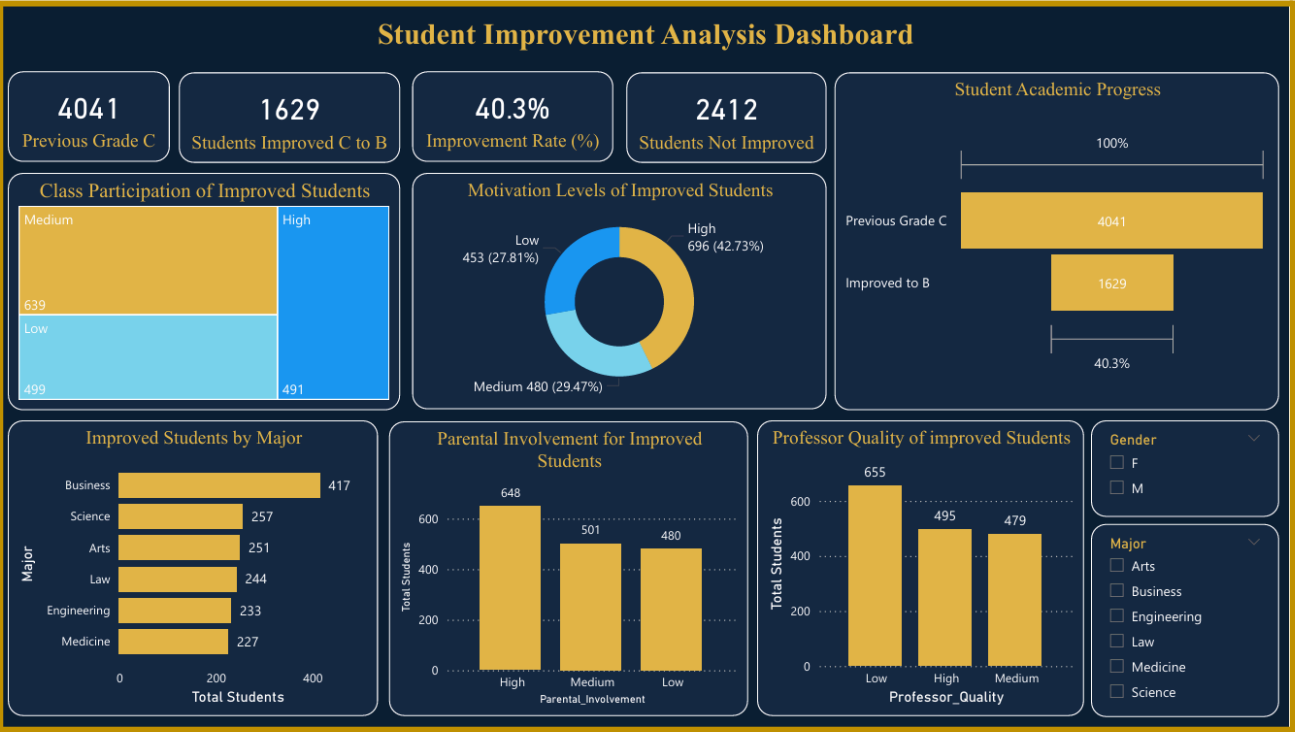
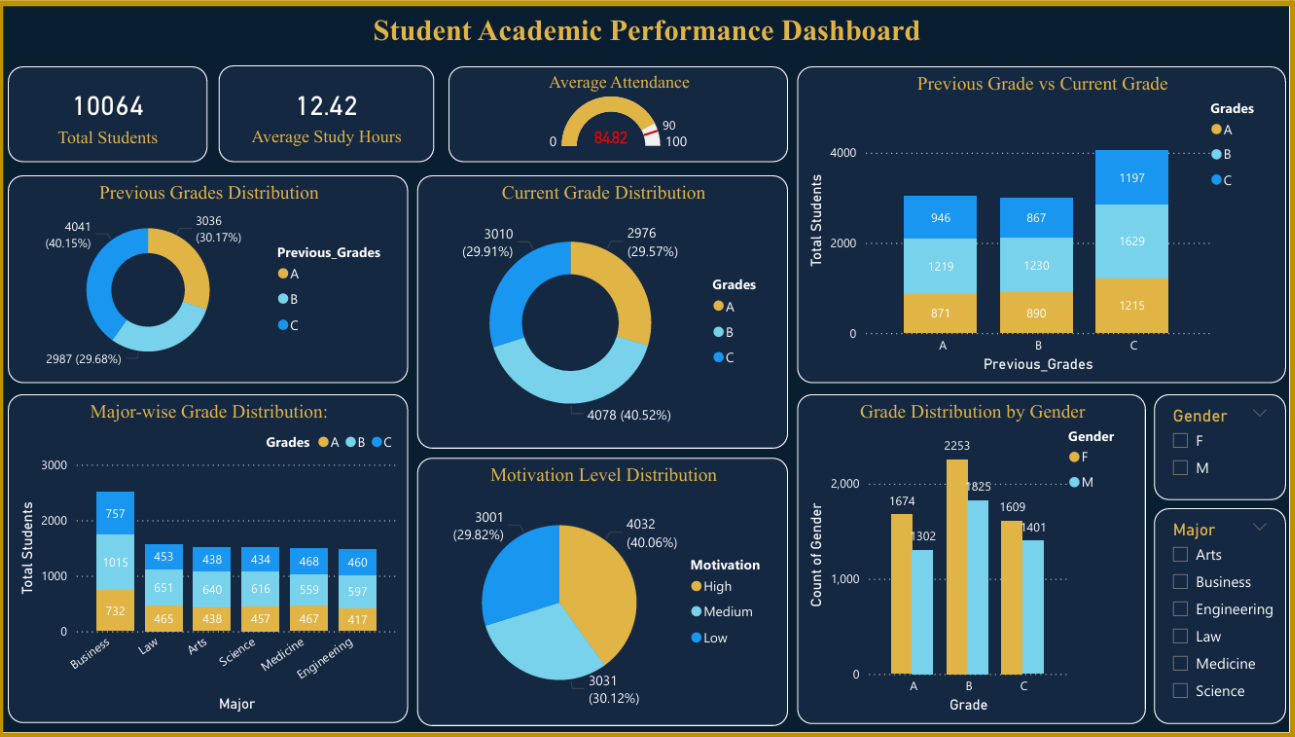
Students Not Improved =

```
CALCULATE(  
    COUNTROWS(student_performance_cleaned),  
    student_performance_cleaned[Previous_Grades] = "C"  
)  
-  
CALCULATE(  
    COUNTROWS(student_performance_cleaned),  
    student_performance_cleaned[Previous_Grades] = "C",  
    student_performance_cleaned[Grades] = "B")
```

2412

Students Not Improved

8. Dashboards (Power BI)



9. Analysis Insights

Descriptive Analysis:

- A total of 10,064 student records were analysed. The average attendance was 84.82%, and students studied an average of 12.42 hours.
- The dataset contains students from different academic backgrounds, majors, school types, and performance levels.
- Most students belonged to the Previous Grade C category, showing that a large portion of students had lower academic performance before the current evaluation period.
- Grade B was the most commonly achieved final grade, indicating that many students moved towards an average academic performance level.
- Gender-wise grade analysis shows similar performance patterns among male and female students, with Grade B being the dominant grade for both groups.
- High levels of motivation, parental involvement, and class participation are observed among a large number of students.
- Attendance levels are generally consistent across students, indicating regular participation in academic activities.
- Students follow relatively stable study habits, with most students studying within a moderate range of hours.
- The distribution of professor quality indicates that students have mixed opinions about teaching quality, with a large portion of students rating it as low.

Diagnostic Analysis:

- The comparison between previous grades and current grades shows that a significant number of students improved from Grade C to Grade B, indicating positive academic progress.
- The improvement funnel confirms that many lower-performing students were able to move to a better performance category.
- Students with high parental involvement appear more frequently among the improved student group, suggesting that parental support contributes to academic improvement.
- Improved students are also associated with higher levels of class participation, indicating that active classroom engagement supports better academic outcomes.
- Higher attendance levels are linked with better academic performance, showing the importance of regular class attendance.

- Students with stronger motivation levels tend to show better academic results and improvement.
- Although many students rated Professor Quality as Low, students still showed improvement, suggesting that personal effort, motivation, attendance, and parental support had a stronger impact on student success.
- Overall, the findings highlight that improving motivation, encouraging active class participation, maintaining good attendance, and increasing parental involvement can contribute significantly to better academic performance and student success.

Future Enhancement:

- Real-time academic performance tracking can be implemented to continuously monitor student progress throughout the academic year.
- Predictive analytics and machine learning models can be developed to identify students who may face academic difficulties and provide early intervention support.
- Additional factors such as learning behaviour, extracurricular activities, and student feedback can be incorporated to gain deeper insights into academic performance.
- Personalized academic recommendations can be generated to help students improve study habits, attendance, and overall performance.
- Faculty and institution-level performance monitoring modules can be added to support data-driven educational planning and policy development.
- Advanced interactive dashboards with automated alerts and reporting features can be implemented to improve decision-making for educators and academic administrators.

10. Conclusion:

The analysis reveals that student academic performance is shaped by multiple academic and personal factors. Although many students initially belonged to the lower-performing Grade C category, a large number successfully improved to Grade B. The improvement analysis highlights the importance of parental involvement, motivation, attendance, class participation and study habits in supporting academic success. These findings can help educational institutions to identify areas for improvement and develop strategies to enhance student learning outcomes and overall academic performance.